

■ Generate a Production-Ready Playwright Test

Act as a Senior SDET with 10+ years of experience.

Generate a Playwright test using JavaScript.

Requirements:

- Use Page Object Model
- Add meaningful assertions
- Use proper waits
- Follow Playwright best practices
- Add comments explaining each step

Scenario:

User logs into the application using valid credentials and verifies dashboard visibility.

■ Convert Manual Test Cases to Playwright

Act as a Playwright Automation Engineer.

Convert the following manual test case into Playwright automation.

Requirements:

- Use Playwright Test framework
- Follow Page Object Model
- Add assertions
- Include negative validations
- Make the code production ready

Manual Test Case:

1. Open Login Page
2. Enter valid username
3. Enter valid password
4. Click Login
5. Verify Dashboard is displayed

■ Generate Robust Locators

Act as a Playwright expert.

Analyze the following HTML.

Generate:

1. Recommended locator
2. Alternative locator
3. Why the locator is stable

Prefer:

- `getByRole()`
- `getByLabel()`
- `getByText()`
- `data-testid`

Avoid brittle XPath.

HTML:

<paste html>

■ Debug a Failed Playwright Test

Act as a Senior Automation Architect.

Analyze the following Playwright failure.

Provide:

1. Root Cause
2. Why it happened
3. Fix
4. Prevention Strategy

Error:

<paste stack trace>

■ ■ Generate POM Structure

Act as a Test Framework Architect.

Design a scalable Playwright framework.

Requirements:

- Page Object Model
- API Layer
- Utilities
- Reporting
- Environment Config
- CI/CD Ready

Generate:

- Folder Structure
- Design Explanation
- Sample Classes

■ Generate API + UI Hybrid Test

Act as a Playwright expert.

Generate a test that:

1. Creates user via API
2. Logs into UI
3. Verifies created user data

Use:

- Playwright APIRequestContext
- Playwright UI automation

Follow best practices.

■ Optimize Slow Playwright Tests

Act as a Playwright performance expert.

Analyze the following test.

Identify:

- Slow operations
- Unnecessary waits
- Locator issues
- Parallelization opportunities

Suggest optimized code.

Code:

<paste code>

■ Generate End-to-End Test Scenarios

Act as a Senior QA Engineer.

For the following feature:

<feature description>

Generate:

- Happy Path
- Negative Scenarios
- Edge Cases
- Security Checks
- Accessibility Checks

Prioritize scenarios for Playwright automation.

■ ■ Generate Visual Testing Strategy

Act as a Test Architect.

Design a visual testing strategy using Playwright.

Include:

- Screenshot comparisons
- Baseline management
- Dynamic content handling
- CI/CD integration

Generate implementation examples.

■■■ Review My Playwright Code Like a Staff Engineer

Act as a Staff SDET.

Review the following Playwright code.

Provide:

1. Code Smells
2. Scalability Issues
3. Flakiness Risks
4. Design Improvements
5. Refactored Version

Code:

<paste code>

■ My Most Used Prompt

Act as a Principal SDET and Playwright expert.

Before generating code:

1. Analyze the requirement.
2. Identify risks.
3. Suggest test scenarios.
4. Design the automation approach.
5. Generate production-ready Playwright code.
6. Explain why the solution is optimal.

Follow Playwright best practices.

Avoid anti-patterns.

Optimize for maintainability and scalability.